



SoC Overview

SHAKTI Group | CSE Dept | PS-CDISHA - RISE LAB

IIT - Madras

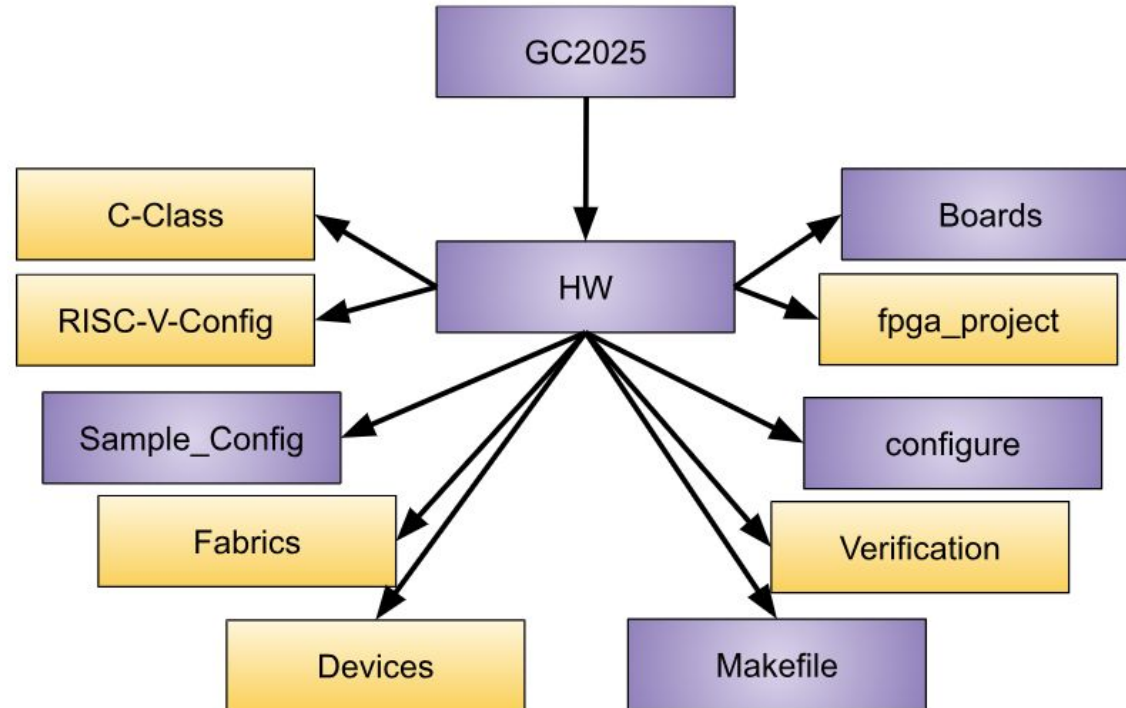
Preview of the Session

1. Introduction
2. Hardware Flow
3. Adding new Boards and IP's
4. Building RTL and Generating Bitstream

Introduction

- Uses Bluespec and Verilog files.
- RTL/Bit stream generation is Makefile based terminal utility.
- Shakti core/peripherals is/are configurable with yaml files.
- Core is built with C-Class & risc-v configure (sample config).
- Cores and peripherals are connected using fabrics.
- Peripheral logics are present in devices.
- Vivado IPs, build process and programming logics are added using tcl scripts.
- Bluespec files for different Shakti variants.
- Standalone Application code for booting.

Hardware Flow



Makefile

- Backbone of overall build utility.
- Whole build process is covered.
- Different switch options are available.
- Does the following.
 - Creates project based on the board name selected.
 - Takes the configuration parameters for Shakti RISC-V Core & Cache.
 - Generates Bsv files into verilog.
 - Add vivado IPs, create project, build the project, program the configuration.
 - e.g. “ **make build BOARD=arty_a7_ganga**”.

Boards

- Board specific files are present for each board/SoC configuration.
- Files covers the
 - Fabrics interconnection.
 - Peripheral IPs & cluster based mapping.
 - Constraints.
 - Top level module mapping with SoC modules and Vivado IPs.
 - TCL files for the IPs and build flow.

Configure & Core Configuration

- Configuration is done by python based configuration in configure directory with yaml files present in sample_config directory.
- Configures the core, CSR, cache, branch predictor and ISA based on the specified configuration and boards LUT factor.
- Separate config files for core, CSR, ISA, FPGA/ASIC, debug, custom changes, etc.
- Configuration details are done with fixed yaml files i.e. c32 and c64 based files for 32-bit and 64-bit cores in Sample Config
- Yaml files configured for a particular SoC using python scripts in configure folder.

C-Class and RISC-V-config

- Ganga/Yamuna are C-Class SoCs.
- C-Class repo is cloned as a set up process.
- Contains architectural part of the core like
 - Pipeline stages,
 - Branch predictor,
 - RISC-V extensions like mbox, fbox, csr, etc.
- Cloned based on the yaml file configuration.



FPGA-project and Verification

FPGA-project:

- This folder will be created by a TCL script while adding vivado IPs/creating a project.
- The Project folder contains vivado dependent folders like the sources, runs and files of the Vivado project and also has a XPR file to have a view of the project in a GUI format.

Verification:

- This directory contains a set of verification tests to check the core with the help of spike.
- There are tests from various categories like compliance, micro-arch level etc.

Devices and Fabrics

Devices:

- Set of peripherals which are developed for SHAKTI.
- These are invoked based on configuration done in SoC & cluster files.
- Few Peripherals available in this repo. UART, I2C, BRAM, DMA, GPIO, SPI, PWM etc.

Fabrics:

- Bridge between core and memory/peripherals.
- Based on use of AXI4 and AXI4lite bus protocols.
- These bus protocols will be present in the Fabrics directory.

Additional directories

Directories	Use
csrbox	Configure the CSR registers in the core.
caches_mmu	Configure the Caches in the core
common_verilog	Contains the pre-requisite verilog files for RTL flow.
common_bsv	Contains the pre-requisite bluespec files for RTL flow.



Adding New Boards:

- For adding new FPGA boards ,a folder needs to be created with a specific board name like arty_a7, consisting of all the necessary files according to the particular board configuration inside the boards folder in HW.
- Then changes in Makefile are required, like adding the board part with a set of if else statements.
- \$BOARD will denote the board name specified here and this should match with the folder name to access files.

```
ifneq ($(BOARD), arty_a7)
FPGA:=xc7a100tcsq324-1
MCS:=true
else ifneq ($(BOARD), arty_a7_yamuna)
FPGA:=xc7a100tcsq324-1
MCS:=true
```

Adding New Boards:

- Adding core configuration for the board which is added in sample_config with either a 32 bit or 64 bit config.
- Another change would be there in Boot files where similar addition is done to generate boot files specific to the board.

Adding New Verilog IP's

- For Adding new Verilog into the core building process, again go for the Makefile and under the division of generate_verilog and specify the path of the verilog file and the verilog file.
- These verilog files are copied to build, while giving “make”.
- So after generation of bitstream, this specific IP will be added to the core.

```
.PHONY: generate_verilog
generate_verilog: $(BSVBUILDDIR)/$(TOP_BIN)
    @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/RegFile.v ${VERILOGDIR}
    @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/BRAM2BEMem.v ${VERILOGDIR}
    @cp ${BS_VERILOG_LIB}/FIFO2.v ${VERILOGDIR}
    @cp ${BS_VERILOG_LIB}/FIFO1.v ${VERILOGDIR}
    @cp ${BS_VERILOG_LIB}/FIFO10.v ${VERILOGDIR}
    @cp ${BS_VERILOG_LIB}/FIFO11.v ${VERILOGDIR}
```



Adding New Bluespec IP's

- For Adding new Bluespec IP's, specify the path of the IP in bsvpath, a file which is present in HW.
- So on compiling these files in bsvpath, newer IP's will be added to the core through bsvpath.
- For looking into integration of any other IP's, please look into the IP_Intergration_with_SHAKTI_SoC manual.

Building core

- We know that the command to build and generate is “make build BOARD=board_name”.
- When this command is given first thing the makefile redirects the process to check whether the prerequisites are available and then proceeds with configuring the core.
- Using “configure”, core configurations for the specific boards are taken and configured according to the XLEN and boards.

Building & Generating verilog and boot files

- Build directory contains the IP's and its verilog files. This will also contain the intermediate folder.
- The Bluespec compiler will compile all the .bsv files involved in the process ,also from bsvpath and will throw errors if there are any.
- Commands like “**make generate_verilog -j**” and “**make generate_boot_files**” will generate boot files & verilog files from the given bluespec library files.



Building IP's and Boards

- There are specific TCL files for automating IP builds for Clock Wizard, Reset, AXI, DDR, XADC etc.
- The command “ make ip_build” invokes the IPs in being called in TCL script. and add them to the project.
- The command “make board_build” will
 - Create vivado project
 - Add the invoked IPs and verilog codes to the project
 - Follow the vivado flow to generate bit stream.
 - Generates the mcs file.

Bitstream and MCS Generation

- Using the TCL script ,a Vivado project is created and initiates a Vivado synthesis and convert these verilog/bluespec code into RTL.
- Also the same RTL which is generated is converted into bitstream with the help of Vivado itself.
- MCS is generated with the help of fpga_top.bit(bitstream) with the help of generate_mcs.tcl which helps in converting this .bit into .mcs.
- .bit file could be found in the c-class.runs folder in the fpga-project directory.
- This MCS file or the Bitstream can be directly programmed to the board either through Vivado or program.tcl and further operations like connecting openOCD and starting a GDB server for debugging the core can be performed.

For Support:

For support please mail your issues into gc2025@shakti.groups.io.

OR

Post Your Issues in the Digital India RISC-V (DIR-V) Grand Challenge-SHAKTI

Whatsapp group.



SHAKTI

Thank you

